

Exercises

Chap 2.1 et Chap 2.2

- These exercises are individual
- Drop the answers here before the **18th of December, midnight [23:59]**

Python

- Explore Covid Data: <https://www.w3resource.com/python-exercises/project/covid-19/index.php> (redo the same in R)
- Loading data: <https://www.w3resource.com/python-exercises/csv/python-csv-exercise-1.php>
- Handle Data: <https://www.w3resource.com/python-exercises/pandas/movies/index.php>

R Prog.

- Handle Dataframes: <https://www.r-exercises.com/2016/11/28/data-frame-exercises-vol-2/>
- Handle Missing Values: <https://www.r-exercises.com/2015/12/14/missing-values/>
- Smooth Data Exploration with `dplyr` : <https://www.r-exercises.com/2017/10/03/dplyr-basics-more-smooth-data-exploration/> (redo the same with another dataset chosen from `data()` command. Ask your own similar questions related to the dataset and answer them)
- Customize Scatterplots: <https://www.r-exercises.com/2016/03/17/customize-scatterplot/> (redo the same with `ggplot2`)
- Boxplot: <https://www.r-exercises.com/2016/09/26/replicating-plots-boxplot-exercises/> (redo the same with `ggplot2`)

Chap 2.3

1. Repeat what is presented in `Streamlit` with `shiny R`
2. Following the content presented here in Part 1. Write a tutorial in `Streamlit` (I'll provide the topic of each tutorial) ⇒ Split the class into groups of 3 (it can be one/two groups of 4) ⇒ Share the group with me and I'll share the

Part 1: Introduction and Basic Programming

Chapter 1: Introduction to Programming and Emerging Technologies

Chapter 2: Basics of Python Programming

Chapter 3: R Programming Basics

Chapter 1: Introduction to Programming and Emerging Technologies

Part 1: Introduction and Basic Programming

Chap 1: Introduction

Introduction (1/2)

Python and R Usage Statistics

Introduction (1/2)

Python and R Loves/Dreaded Statistics

Introduction (1/2)

Python and R Jobs Statistics

Introduction (1/2)

Python and R in DS Statistics

Introduction (1/2)

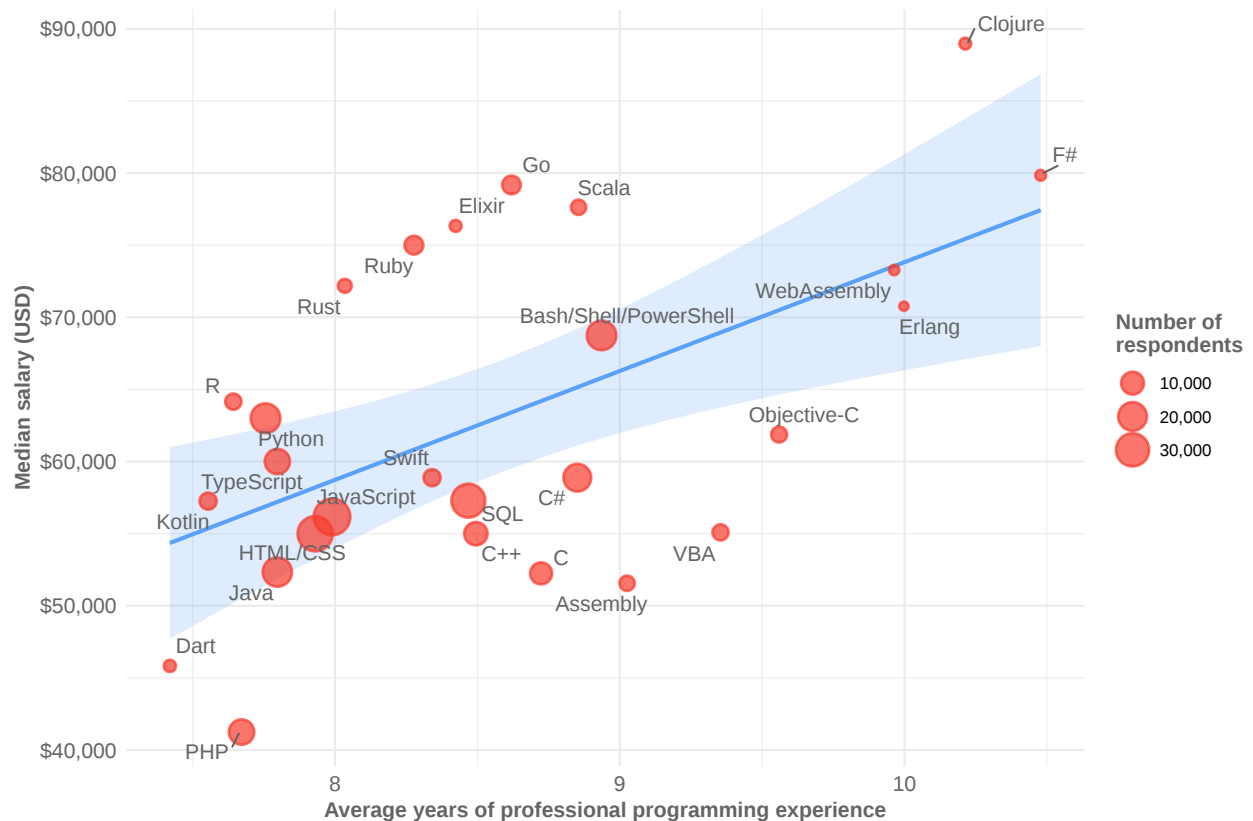
Python and R in DS Statistics

Introduction (1/2)

Python and R Salary Statistics

Introduction (1/2)

Python and R Salary Statistics (older)



Introduction (2/2)

Other stats

- Programming stats <https://insights.stackoverflow.com/survey>
- Data science stats: <https://www.datacamp.com/blog/python-vs-r-for-data-science-whats-the-difference>
- comparison
<https://s3.amazonaws.com/assets.datacamp.com/email/other/Python+vs+R.pdf>

Setting up Python and R

- Anaconda (Python) <https://www.anaconda.com/products/distribution>
- Jupyter Notebooks: <https://jupyter.org/install>
- R: <https://cran.r-project.org/>
- RStudio: <https://www.rstudio.com/products/rstudio/download/>

Practice - Environment Setup

- Basic task: Type `1 + 2` in both Python and R environments.

Draft

Chapter 2: Basics of Python Programming

Part 1: Introduction and Basic Programming

Chap 1: Introduction

Chap. 2: Basics of Python Programming

Hello World Program in Python

- **Definition** : The "Hello World" program is a simple script that displays "Hello, World!" on the screen. It's a traditional way to start learning a new programming language.
- **Syntax** : `print("Hello, World!")`
- **Example** :

```
print("Hello, World!")
```

Python Variables

- **Definition** : Variables are used to store data values. In Python, a variable is created the moment you first assign a value to it.
- **Naming Rules for Python Variables** :
 - Must start with a letter or the underscore character
 - Cannot start with a number
 - Can only contain alphanumeric characters and underscores (A-z, 0-9, and _)
 - **Case-sensitive** (age, Age, and AGE are three different variables)

- **Syntax :** `variable_name = value`
- **Example :**

```
x = 5
name = "Alice"
print(x, name)
```

Python Data Types (1/2) - Introduction

- **Definition :** Data types are the classification or categorization of data items. Python has various data types including Integer, Float, String, List, Tuple, and Dictionary.
- **Main Data Types :**
 - Integer, Float, String, List, Tuple, Dictionary, Set, Boolean
- **Memory Size :**
 - The size in memory varies depending on the data type and the value it holds.
- **Example :**

```
integer_var = 10
float_var = 20.5
string_var = "Hello"
```

Python Data Types (2/2) - Literals

- **Integer :** Whole numbers, e.g., `5`
- **Float :** Decimal point numbers, e.g., `5.0`
- **String :** Sequence of characters, e.g., `"Hello"`
- **List :** Ordered and changeable collection, e.g., `[1, 2, 3]`
- **Tuple :** Ordered and unchangeable collection, e.g., `(1, 2, 3)`
- **Dictionary :** Collection with key-value pairs, e.g., `{"name": "John", "age": 30}`

- **Set** : Unordered collection of unique elements, e.g., `{1, 2, 3}`
- **Boolean** : Represents `True` or `False`

Python Operators (1/3) - Introduction

- **Definition** : Operators are used to perform operations on variables and values. Python has arithmetic, comparison, assignment, logical, identity, membership, and bitwise operators.
- **Example** :

```
# Arithmetic Operator
a = 10
b = 3
print(a + b) # Addition
print(a * b) # Multiplication
```

Python Operators (2/3) - Overview

- **Definition** : Operators are special symbols in Python that carry out arithmetic or logical computation.
- **Types of Operators** :
 - Arithmetic Operators
 - Comparison (Relational) Operators
 - Assignment Operators
 - Logical Operators
 - Identity Operators
 - Membership Operators
 - Bitwise Operators

Python Operators (3/3) - Details

- **Arithmetic Operators** : `+`, `-`, `/`, `%`, `*`, `//`, `**`
- **Comparison Operators** : `==`, `!=`, `>`, `<`, `>=`, `<=`

- **Assignment Operators** : `=`, `+=`, `-=`, `*=`, `/=`, etc.
- **Logical Operators** : `and`, `or`, `not`
- **Identity Operators** : `is`, `is not`
- **Membership Operators** : `in`, `not in`
- **Bitwise Operators** : `&`, `|`, `^`, `~`, `<<`, `>>`
- **PEMDAS Rule** : Parentheses, Exponents, Multiplication/Division, Addition/Subtraction.

Python Print Function

- **Definition** : The print function in Python is used to output data to the standard output device (screen).
- **Syntax** : `print(object(s), sep=separator, end=end, file=file, flush=flush)`
- **Example** :

```
print("Hello, Python!")
print("A", "B", "C", sep="-")
```

Python Input Function

- **Definition** : The input function allows the user to take input from the keyboard as a string.
- **Syntax** : `input([prompt])`
- **Note** : The output of the input function is always a string. It may need to be converted (cast) into the appropriate type.
- **Example** :

```
name = input("Enter your name: ")
print("Hello", name)
```

```
age = input("Enter your age: ")
```

```
age = int(age) # Casting string to integer
```

Python Casting

- **Definition** : Casting in Python is when you convert a variable value from one type to another.
- **Types of Casting** :
- `int()` : Converts to an integer
- `float()` : Converts to a float
- `str()` : Converts to a string
- **Example** :

```
x = "123"  
y = int(x)  
print(y, type(y))
```

Python If Statements (1/4) - If

- **Definition** : An 'if statement' is used to test a specific condition. If the condition is true, a block of code will be executed.
- **Syntax** :

```
if condition:  
    # code to execute
```

- **Example** :

```
x = 10  
if x > 5:  
    print("x is greater than 5")
```

Python If Statements (2/4) - If-else

- **Definition:** An 'if-else' statement checks a condition and executes a block of code if the condition is true, otherwise, it executes a different block of code.
- **Syntax:**

```
if condition:
    # code if true
else:
    # code if false
```

- **Example :**

```
age = 20
if age >= 18:
    print("Adult")
else:
    print("Not Adult")
```

Python If Statements (3/4) - If-Elif-Else

- **Definition :** The 'elif' (else if) is used to include multiple conditional expressions.
- **Syntax :**

```
if condition1:
    # code if condition1 is true
elif condition2:
    # code if condition2 is true
else:
    # code if none of the conditions are true
```

- **Example :**

```
age = 15
if age < 13:
```

```
    print("Child")
elif age < 18:
    print("Teenager")
else:
    print("Adult")
```

Python If Statements (4/4) - Ternary Operator

- **Definition** : Python ternary operators are inline if-else statements.
- **Syntax** : `[on_true] if [condition] else [on_false]`
- **Example** :

```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status)
```

Python Loops (1/4) - Introduction

- **Definition** : Loops in Python are used to execute a block of code repeatedly.
- **For Loop** : Iterates over a sequence.
- **While Loop** : Repeats as long as a certain boolean condition is met.
- **Example** :

```
# For Loop
for i in range(5):
    print(i)

# While Loop
count = 0
while count < 5:
    print(count)
    count += 1
```

Python Loops (2/4) - Break

- **Definition** : The 'break' statement is used to **exit the loop entirely**.
- **Syntax** : `break`
- **Example** :

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

Python Loops (3/4) - Continue

- **Definition** : The 'continue' statement **skips the current iteration** and continues with the next.
- **Syntax** : `continue`
- **Example** :

```
for i in range(10):  
    if i == 5:  
        continue  
    print(i)
```

Python Loops (4/4) - Break vs Continue

- **Comparison** :
- **Break** : Exits the loop when a certain condition is met.
- **Continue** : Skips part of the loop when a certain condition is met, but continues the rest of the loop.
- **Example** :

```
for i in range(10):  
    if i == 5:
```

```
        continue # Skip 5
    elif i == 8:
        break # Exit loop at 8
    print(i) # Will print 0 to 4, 6, 7
```

Python Functions (1/3) - Basics

- **Definition** : A function is a block of code that only runs when it is called.
- **Syntax** :

```
def function_name(parameters):
    # code
```

- **Example** :

```
def greet(name):
    print("Hello, " + name)

greet("Alice")
```

Python Functions (2/3) - Types

- **Types** :
 1. **With Parameters** : Functions that take arguments.
 2. **Without Parameters** : Functions that do not take arguments.
 3. **Return Value** : Functions that return a value.
 4. **No Return Value** : Functions that perform an action but do not return a value.
- **Examples** :

```
# With Parameters
def add(a, b):
    return a + b
```

```
# Without Parameters
def say_hello():
    print("Hello")
```

Python Functions (3/3) - Advanced Tricks

- **Intermediate/Advanced Tricks :**
- **Default Parameter Values :** `def function_name(parameter=value):`
- **Keyword Arguments :** `function_name(param1=value1, param2=value2)`
- **Arbitrary Arguments :** `def function_name (*args):`
- **Arbitrary Keyword Arguments :** `def function_name (**kwargs):`
- **Examples :**

```
# Default Parameter Values
def greet(name="World"):
    print("Hello, " + name)

greet("Alice")
greet() # Uses default value "World"

# Keyword Arguments
def full_name(first, last):
    print(first + " " + last)

full_name(last="Doe", first="John")

# Arbitrary Arguments
def add_numbers(*numbers):
    total = 0
    for num in numbers:
        total += num
    return total

print(add_numbers(1, 2, 3, 4))
```



```
# Arbitrary Keyword Arguments
def person_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

person_info(name="Alice", age=30, city="New York")
```

Python Lists

- **Definition :** A list is a collection that is ordered and changeable. Lists are written with square brackets.
- **Syntax :** `list_name = [item1, item2, item3]`
- **Example :**

```
fruits = ["apple", "banana", "cherry"]
print(fruits[1]) # Outputs 'banana'
```

Python Dictionaries

- **Definition :** A dictionary is a collection which is unordered, changeable, and indexed. In Python, dictionaries are written with curly brackets, and they have keys and values.
- **Syntax :** `dictionary_name = {key1: value1, key2: value2}`
- **Example :**

```
person = {"name": "John", "age": 30}
print(person["name"]) # Outputs 'John'
```

Python Tuples

- **Definition :** A tuple is a collection which is ordered and unchangeable. Tuples are written with round brackets.
- **Syntax :** `tuple_name = (item1, item2, item3)`

- **Example :**

```
coordinates = (4, 5)
print(coordinates[0]) # Outputs '4'
```

Python DataFrames

- **Definition :** DataFrame is a 2-dimensional labeled data structure with columns of potentially different types. It's similar to a spreadsheet or SQL table.
- **Syntax :** `import pandas as pd` `df = pd.DataFrame(data)`
- **Example :**

```
import pandas as pd
data = {'Name': ['John', 'Anna'], 'Age': [28, 22]}
df = pd.DataFrame(data)
print(df)
```

Mutable vs Immutable in Python

- **Definition :**
- Mutable objects can change their value but keep their id() (e.g., list, dictionary).
- Immutable objects can't change their value (e.g., int, string, tuple).
- **Example :**

```
# Mutable
my_list = [1, 2, 3]
my_list.append(4) # Changes the content without changing the id
print(my_list)

# Immutable
my_num = 10
```

```
my_num += 5  # Creates a new object with a different id
print(my_num)
```

Python Cheatsheet

- Python Cheatsheet: <https://www.datacamp.com/community/tutorials/python-data-science-cheat-sheet-basics>

Practice - Python Exercises

- Python Exercises for Beginners: <https://www.w3resource.com/python-exercises/>

Comparison Between R and Python

- Comparison Guide of R and Python:
<https://www.datacamp.com/community/tutorials/r-or-python-for-data-analysis>

Chapter 3: R Programming Basics

Part 1: Introduction and Basic Programming

Chap 1: Introduction

Chap. 2: Basics of Python Programming

Chap. 3: Basics of Python Programming

Hello World Program in R

- **Definition:** The "Hello World" program in R is a simple script that displays "Hello, World!" on the screen. It's commonly used as the first step in learning a new programming language.
- **Syntax :** `print("Hello, World!")`
- **Example :**

```
print("Hello, World!")
```

R Variables

- **Definition:** R variables store data values. Based on the value assigned, R automatically determines the variable's data type.
- **Syntax :** `variable_name <- value`
- **Example :**

```
x <- 5  
name <- "Alice"
```

```
print(x)
print(name)
```



It's possible to use the `=` symbol as well

R Naming Rules for Variables

- **Naming Rules :**

1. Should start with a letter or a **dot** (but a dot cannot be followed by a number).
2. Can contain letters, numbers, **dots**, and underscore characters.
3. Case-sensitive (`age`, `Age`, and `AGE` are different variables).

R Data Types - Introduction

- **Definition:** Data types in R represent the kind of data you can store in a variable.
- **Main Data Types :**
 - Numeric, Integer, Complex, Logical, Character

R Data Types - Details

- **Numeric:** Any number with or without decimal, e.g., `5`, `12.5`.
- **Integer:** Whole numbers, e.g., `5L` (the `L` tells R to store this as an integer).
- **Complex:** Complex numbers, e.g., `1+4i`.
- **Logical:** Boolean values, `TRUE` or `FALSE` or `T` or `F`
- **Character:** Text/string data, e.g., `"Hello"`.

R Operators - Overview

- **Definition:** Operators are symbols that tell R to perform specific mathematical or logical manipulations.
- **Types of Operators :**

- Arithmetic
- Relational
- Logical
- Assignment

R Operators - Details

- **Arithmetic Operators :** `+`, `-`, `*`, `/`, `^`, `%%`, `/%`.
- **Relational Operators :** `==`, `!=`, `>`, `<`, `>=`, `<=`.
- **Logical Operators :** `&`, `|`, `!`, `&&`, `||`.
- **Assignment Operators:** `<-`, `=`, `>`.

R Print Function

- **Definition:** The print function in R is used to display the value of a variable or a result of a computation.
- **Syntax :** `print(object)`
- **Example :**

```
print("Hello, R!")
```

R Input Function

- **Definition:** Reading user input in R is done using the `readline` function.
- **Syntax :** `input <- readline(prompt)`
- **Example :**

```
name <- readline("Enter your name: ")
print(paste("Hello", name))
```

R Casting

- **Definition:** Casting in R involves converting the type of a variable from one to another, such as converting a character to an integer.

- **Types of Casting :**

- `as.integer()`
- `as.numeric()`
- `as.character()`

- **Example :**

```
x <- "123"  
y <- as.integer(x)  
print(y)
```

R If Statements

- **Definition:** The 'if statement' in R is used for conditional execution of code.

- **Syntax :**

```
if (condition) {  
    # code to execute if condition is true  
}
```

- **Example :**

```
x <- 10  
if (x > 5) {  
    print("x is greater than 5")  
}
```

R If-Else and If-Elseif-Else Statements

- **If-Else Syntax :**

```
if (condition) {  
  # code if true  
} else {  
  # code if false  
}
```

- **If-Elseif-Else Syntax :**

```
if (condition1) {  
  # code if condition1 is true  
} else if (condition2) {  
  # code if condition2 is true  
} else {  
  # code if none of the conditions are true  
}
```

R Ternary Operator

- **Definition :**

Although R doesn't have a traditional ternary operator like some other languages, the `ifelse` function can achieve similar functionality.

- **Syntax :** `ifelse(test, yes, no)`

- **Example :**

```
age <- 20  
status <- ifelse(age >= 18, "Adult", "Minor")  
print(status)
```



`ifelse` and `if_else` are two that do the same thing but are not from the same library (`ifelse` is a built-in R function whereas `if_else` is from `dplyr`)

R Loops - Break and Continue

- **Break:** Used to exit from a loop entirely.
- **Continue:** In R, there's no direct equivalent to a 'continue' statement, but similar behavior can be achieved using an `if` statement.
- **Break Example :**

```
for (i in 1:10) {  
  if (i == 6) {  
    break  
  }  
  print(i)  
}
```

- **'Continue' Example :**

```
for (i in 1:10) {  
  if (i == 6) {  
    next  
  }  
  print(i)  
}
```

R Functions - Basics and Types

- **Definition:** Functions in R are used to encapsulate a set of operations or procedures to perform a specific task.
- **Types :**
 1. **With Parameters:** Functions that take arguments.
 2. **Without Parameters:** Functions that do not take arguments.
 3. **Return Value:** Functions that return a value.
 4. **No Return Value:** Functions that perform actions without returning a value.
- **Basic Syntax :**

```
functionName <- function(arg1, arg2) {  
  # function body  
}
```

- **Example :**

```
greet <- function(name) {  
  print(paste("Hello", name))  
}  
greet("Alice")
```

R Functions - Advanced Concepts

- **Advanced Concepts :**

- **Default Parameter Values:** Specifying default values for function arguments.
- **Returning Multiple Values:** Using lists to return multiple values.
- **Anonymous Functions:** Functions without a name and often used in apply functions.

- **Examples :**

```
# Default Parameter Values  
greet <- function(name="World") {  
  print(paste("Hello", name))  
}  
greet()  
greet("Alice")  
  
# Returning Multiple Values  
calculate <- function(x, y) {  
  sum <- x + y  
  product <- x * y  
  return(list(Sum = sum, Product = product))  
}
```

```
}  
result <- calculate(10, 5)  
print(result)  
  
# Anonymous Functions  
nums <- c(1, 2, 3, 4, 5)  
squares <- sapply(nums, function(x) x^2)  
print(squares)
```



Today, we mainly use `purrr` library for all apply operations with `map` functions

R Cheatsheet

- R Cheatsheet: <https://rstudio.com/wp-content/uploads/2016/10/r-cheatsheet-3.pdf>

Practice - R Exercises

- R Exercises for Beginners: <https://www.r-exercises.com/start-here-to-learn-r/>

Comparison Between R and Python

- Comparison Guide of R and Python:
<https://www.datacamp.com/community/tutorials/r-or-python-for-data-analysis>

Part 2: Data Manipulation and Visualization

Chapter 1: Data Manipulation and data cleaning

Chapter 2: Data Visualization

Chapter 3: Web-viewer for Dash-boarding

Exercises

Chapter 1: Data Manipulation and data cleaning

Part 2: Data Manipulation and Visualization Breakdown

Introduction to Data Cleaning and Manipulation - Definition

- **Definition** : Data cleaning and manipulation involve preparing raw data for analysis by making it clean and consistent throughout. This includes handling missing values, removing duplicates, and correcting errors.
- **Python & R Example** :
 - Python (Pandas): `df.drop_duplicates()`
 - R (Tidyverse): `distinct(df)`
- **Reference** :
 - Python Pandas: [Pandas Documentation](#)
 - R Tidyverse: [Tidyverse Documentation](#)

Importance of Data Cleaning and Manipulation

- **Importance** : Ensures the accuracy and completeness of the data for analysis, which is crucial for making informed decisions based on the data.
- **Python & R Example** :
 - Python: Checking for null values with `df.isnull().sum()`
 - R: Checking for missing values with `sum(is.na(df))`
- **Reference** :
 - Data Cleaning: [Towards Data Science Article](#)

Overview of the Data Preparation Process

- **Overview** : The data preparation process typically involves data collection, cleaning, manipulation, and analysis. It sets the foundation for any data analysis or machine learning project.
- **Python & R Example** :
 - Python: `df.info()` for a quick overview of the dataset
 - R: `glimpse(df)` from the `dplyr` package for a similar overview

```
Rows: 150
Columns: 5
$ Sepal.Length <dbl> 5.1, 4.9, 4.7, 4.6, 5.0, 5.4...
$ Sepal.Width <dbl> 3.5, 3.0, 3.2, 3.1, 3.6, 3.9...
$ Petal.Length <dbl> 1.4, 1.4, 1.3, 1.5, 1.4, 1.7...
$ Petal.Width <dbl> 0.2, 0.2, 0.2, 0.2, 0.2, 0.4...
$ Species <fct> setosa, setosa, setosa, seto...
```

- **Reference** :
 - Data Preparation Process: [Data Preparation Guide](#)

Data Types and Structures Review - Numerical, Categorical, Date/Time, and Text Data

- **Definition** : Understanding different data types (numerical, categorical, date/time, text) is crucial for selecting appropriate analysis and data manipulation techniques.
- **Python & R Example** :
 - Python: `df.dtypes` to check data types
 - R: `str(df)` to check data structures and types
- **Reference** :
 - Data Types: [Understanding Data Types](#)

Data Structures (Definition)

- **Definition :** Data structures are formats for organizing, managing, and storing data in a way that facilitates access and modifications.
- **Python & R Example :**
 - Python: List, Dictionary, DataFrame in Pandas
 - R: Vector, List, DataFrame in base R and Tidyverse
- **Reference :**
 - Python Data Structures: [Python Documentation](#)
 - R Data Structures: [R Data Structures](#)

Vectors

- **Definition :** Vectors are a basic data structure that store elements of the same type. In R, they are fundamental, while Python uses lists for similar purposes.
- **Python & R Example :**
 - Python: `[1, 2, 3]` (List as vector)
 - R: `c(1, 2, 3)`
- **Reference :**
 - Python Lists: [Python Lists](#)
 - R Vectors: [R Vectors](#)

Lists

- **Definition :** Lists are ordered collections that can hold a variety of data types. In Python, lists are versatile, while in R, lists can contain different types of elements, including vectors and other lists.
- **Python & R Example :**
 - Python: `[1, "a", 3.14]`
 - R: `list(1, "a", 3.14)`
- **Reference :**
 - Python Lists: [Python Lists](#)

- R Lists: [R Lists](#)

Matrices

- **Definition** : Matrices are two-dimensional, rectangular layouts of elements. Unlike data frames, matrices contain elements of the same data type.
- **Python & R Example** :
 - Python: `numpy.array([[1, 2], [3, 4]])` using NumPy
 - R: `matrix(c(1, 2, 3, 4), nrow=2, ncol=2)`
- **Reference** :
 - Python NumPy: [NumPy Quickstart](#)
 - R Matrices: [R Matrices](#)

Data Frames

- **Definition**: Data frames are two-dimensional structures with columns that can hold different types of data. They are analogous to SQL tables or spreadsheets.
- **Python & R Example** :
 - Python: `pandas.DataFrame(data={'Age': [12, 25], 'Nom': ["Adlphe", "Hitler"]})`
 - R: `data.frame(A=c(1, 2), B=c(3, 4))`
- **Reference** :
 - Pandas DataFrame: [Pandas DataFrame](#)
 - R Data Frames: [R Data Frames](#)

Basic Statistics for Data Understanding

- **Definition**: Basic statistics provide a foundational understanding of the data's central tendency, dispersion, and shape.
- **Python & R Example** :
 - Python (Pandas): `df.describe()` for summary statistics

- R (dplyr): `summary(df)` for summary statistics

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
Min. :4.300	Min. :2.000	Min. :1.000	Min. :0.100	setosa :50
1st Qu.:5.100	1st Qu.:2.800	1st Qu.:1.600	1st Qu.:0.300	versicolor:50
Median :5.800	Median :3.000	Median :4.350	Median :1.300	virginica :50
Mean :5.843	Mean :3.057	Mean :3.758	Mean :1.199	
3rd Qu.:6.400	3rd Qu.:3.300	3rd Qu.:5.100	3rd Qu.:1.800	
Max. :7.900	Max. :4.400	Max. :6.900	Max. :2.500	

- **Reference :**

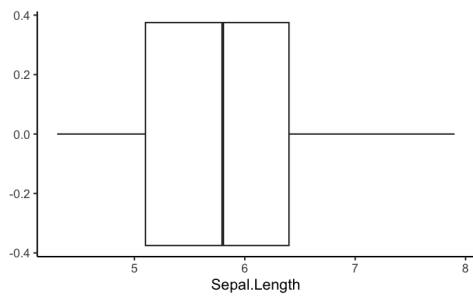
- Python Pandas: [Pandas Describe](#)
- R dplyr: [R Summary](#)

Identifying Missing Values

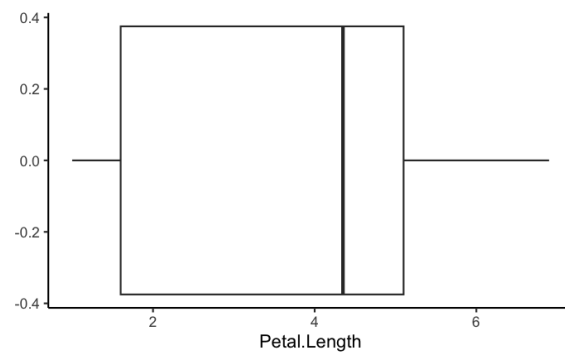
- **Definition:** Identifying missing values is crucial for determining the next steps in data cleaning and preparation.
- **Python & R Example :**
 - Python: `df.isnull().sum()`
 - R: `sum(is.na(df))`
- **Reference :**
 - Handling Missing Data in Python: [Pandas Documentation](#)
 - Handling Missing Data in R: [NA Handling in R](#)

Identifying Outliers

- **Definition :** Outliers are data points that differ significantly from other observations. Identifying outliers is essential for accurate analysis.
- **Python & R Example :**
 - Python: Using Pandas and Seaborn - `sns.boxplot(x=df['column'])`
 - R: Using ggplot2 - `ggplot(df, aes(x=column)) + geom_boxplot()`



```
ggplot(df, aes(x=Sepal.Length)) +  
geom_boxplot() + theme_classic()
```



```
ggplot(df, aes(x=Petal.Length)) + geom_boxplot()  
+ theme_classic()
```

- **Reference :**

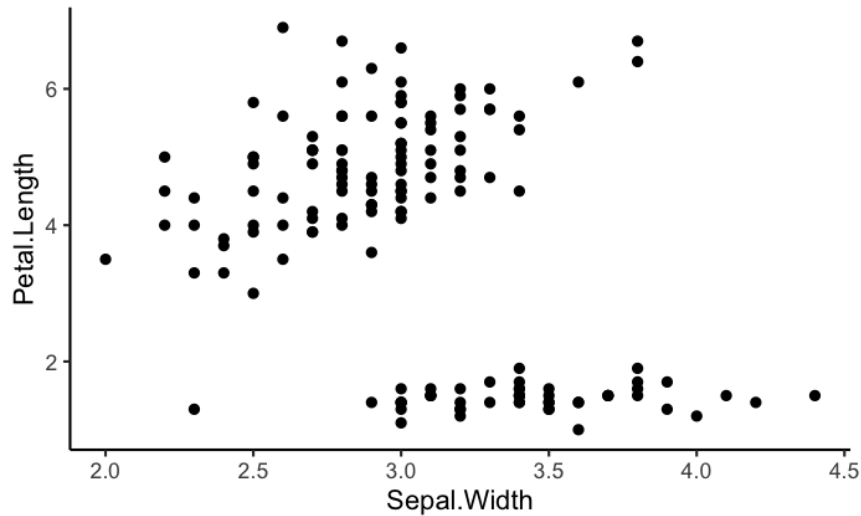
- Python Seaborn: [Seaborn Boxplot](#)
- R ggplot2: [ggplot2 Boxplot](#)

Data Visualization for Anomaly Detection

- **Definition :** Visual techniques for detecting anomalies can help in identifying patterns, outliers, and unusual data points.

- **Python & R Example :**

- Python: `sns.scatterplot(x='column1', y='column2', data=df)`
- R: `ggplot(df, aes(x=column1, y=column2)) + geom_point()`



```
ggplot(df, aes(x = Sepal.Width, y=Petal.Length)) +  
  geom_point() +  
  theme_classic()
```

- **Reference :**

- Python Visualization: [Seaborn Scatterplot](#)
- R Visualization: [ggplot2 Scatter](#)

Techniques for Handling Missing Data - Definitions

- **Definition:** Techniques for handling missing data include deletion, imputation, and interpolation.
- **Python & R Example :**
 - Deletion: Removing rows with missing values.
 - Imputation: Replacing missing values with statistical measures (mean, median).
 - Interpolation: Estimating missing values using other data points.

- **Reference :**

- Python Data Cleaning: [Handling Missing Data in Pandas](#)
- R Data Cleaning: [Imputing Missing Data in R](#)

Handling Missing Data - Deletion

- **Python Example :** `df.dropna()`
- **R Example :** `na.omit(df)`
- **Considerations:** Deletion is simple but can lead to significant data loss, especially if the dataset is not large.

Handling Missing Data - Imputation

- **Python Example :** `df.fillna(df.mean())`
- **R Example :** `library(mice); mice(df, m=1, method='mean')`
- **Considerations:** Imputation preserves data size but can introduce bias.

Handling Missing Data - Interpolation

- **Python Example:** `df.interpolate()`
- **R Example:** `na.approx(df)`
- **Considerations:** Interpolation is useful for time-series data, making assumptions about the relationship between missing points.

Decision Criteria for Choosing Appropriate Methods

- **Factors to Consider :**
 - Data size and structure
 - The proportion of missing data
 - The assumed mechanism of missingness (MCAR, MAR, NMAR)
 - The potential impact on analysis

Data Transformation - Define Data Transformation

- **Definition :** Data transformation involves changing the scale, distribution, or form of data to meet analytical needs or improve model performance.
- **Python & R Considerations :**
 - Python (Pandas, Scikit-learn) and R (dplyr, tidyr) offer various functions and packages for efficient data transformation.

Normalization and Standardization

- **Normalization:** Rescaling data to a [0, 1] range.
 - **Python Example:** `from sklearn.preprocessing import MinMaxScaler`
 - **R Example :** `library(scales); df_scaled <- scale(df, center=FALSE, scale=range(df))`
- **Standardization:** Rescaling data to have a mean of 0 and a standard deviation of 1.
 - **Python Example:** `from sklearn.preprocessing import StandardScaler`
 - **R Example :** `scale(df)`

Encoding Categorical Data

- **Definition :** Encoding transforms categorical data into a numerical format to be processed by algorithms.
- **Python & R Example :**
 - Python: `pd.get_dummies(df['column'])`
 - R: `model.matrix(~column-1, data=df)`

Date and Time Data Transformations

- **Definition :** Transforming date and time data involves converting these values into a format or structure suitable for analysis or feature engineering.
- **Python & R Example :**
 - Python: `pd.to_datetime(df['date_column'])`
 - R: `as.Date(df$date_column)`

Chapter 2: Data Visualization

Part 2: Data Manipulation and Visualization Breakdown

Introduction to Data Visualization - Definition

- **Definition:** Data Visualization involves representing data in a graphical format to provide an accessible way to see and understand trends, outliers, and patterns.
- **Example :**
 - Python (Matplotlib): `plt.hist(data)`
 - R (ggplot2): `ggplot(data) + geom_histogram()`
- **Reference :**
 - Python Matplotlib: [Matplotlib Documentation](#)
 - R ggplot2: [ggplot2 Documentation](#)

Importance of Visualization

- **Importance:** Visualization simplifies complex information, enhances understanding through visuals, facilitates quick decision-making, and helps identify trends and outliers.
- **Use Cases :**
 1. Simplifying complex datasets into understandable visuals.
 2. Enhancing storytelling with data for better communication.
 3. Facilitating quicker decision-making processes.
 4. Identifying trends, patterns, and outliers effectively.

Why Visualization is Crucial in Data Analysis

- **Crucial Factors :**

1. Visualization makes data more accessible and interpretable to non-experts.
2. It supports exploratory data analysis to uncover hidden patterns.
3. Visuals help in validating hypotheses and assumptions.
4. It is essential for effective data communication and presentation.

Design Principles for Data Visualization

- **Design Principles :**
 - Clarity, simplicity, and a focus on the message are key.
 - Choose the right chart type based on what you want to convey.
- **Good vs. Bad Plot :**
 - Show examples of a well-designed plot versus a cluttered and confusing plot.
- **Reference :** [Visualizing Data: Tips and Tricks](#)

Charts and Graphs Basics

- **Basics :** Overview of basic chart types - Bar, Line, Pie, Scatter, Histogram

<https://datavizproject.com/>

- **Reference :** [Data Visualization 101](#)

Advanced Visualizations - Maps and More

- **Types :**
 - Geographic Maps, Heat Maps, and Choropleth Maps for spatial data representation.
 - Advanced visualizations include 3D plots, network graphs, and interactive charts.
- **Reference:** [Advanced Data Visualization Techniques](#)

Storytelling with Data Visualization

- **Storytelling :**
 - Combining infographics and dashboards to tell compelling stories with data.
 - Emphasizes the narrative structure around data to engage and inform.
- **Reference:** [Storytelling with Data](#)

Tools and Software for Data Visualization

- **Overview :**
 - Commercial Tools: Tableau, Power BI, Qlik.
 - Open-Source Tools: D3.js, Python libraries (Matplotlib, Seaborn, Plotly), R packages (ggplot2, shiny).
- **Reference:** [Overview of Data Visualization Tools](#)

Creating Plots in Python and R Using the Iris Dataset

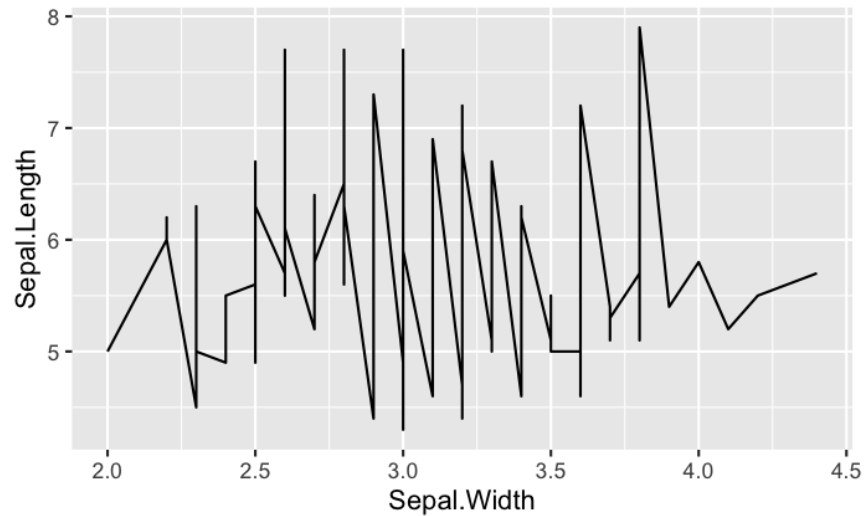
Line Chart - Sepal Width vs. Sepal Length

- **Python (Seaborn) :**

```
import seaborn as sns
sns.lineplot(data=iris,
              x="sepal_width",
              y="sepal_length")
```

- **R (ggplot2) :**

```
ggplot(iris,
        aes(
            x=sepal_width,
            y=sepal_length
        )) +
  geom_line()
```

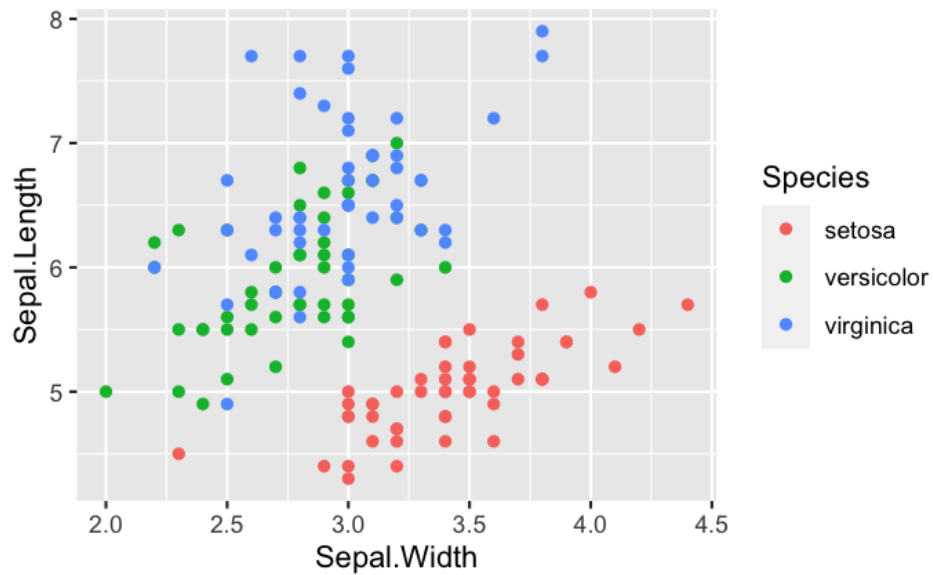
Scatter Plot - Sepal Width vs. Sepal Length by Species

- **Python (Seaborn) :**

```
sns.scatterplot(data=iris,  
                 x="sepal_width",  
                 y="sepal_length",  
                 hue="species")
```

- **R (ggplot2) :**

```
ggplot(iris,  
       aes(x=sepal_width,  
           y=sepal_length,  
           color=species)) +  
  geom_point()
```



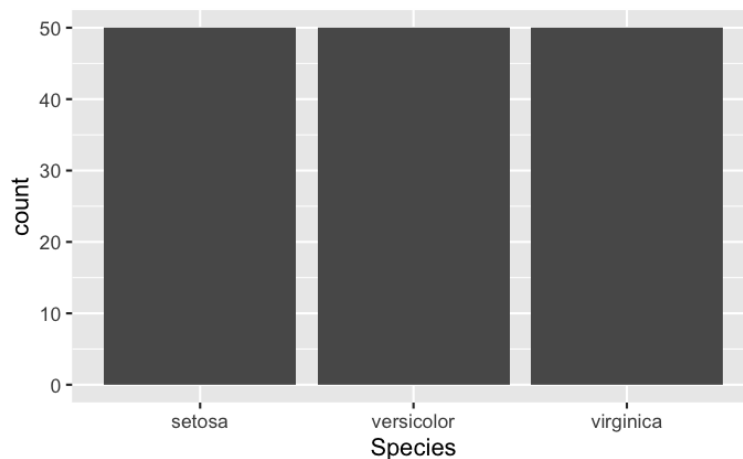
Bar Chart - Species Count (categorical variables)

- **Python (Seaborn) :**

```
sns.countplot(x="species", data=iris)
```

- **R (ggplot2) :**

```
ggplot(iris, aes(x=species)) +  
  geom_bar()
```



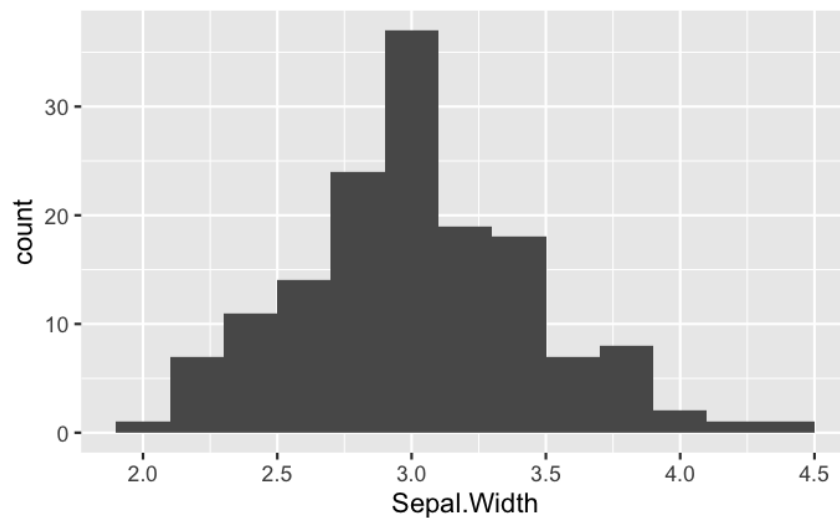
Histogram - Sepal Width Distribution

- Python (Seaborn) :

```
sns.histplot(data=iris, x="sepal_width", kde=True)
```

- R (ggplot2) :

```
ggplot(iris,  
       aes(x=sepal_width)) +  
  geom_histogram(binwidth=0.2)
```



Which one to choose 🤔

<https://www.data-to-viz.com/>

Chapter 3: Web-viewer for Dash-boarding

Part 2: Data Manipulation and Visualization

Chap. 3: Web-viewer for Dash-boarding

Streamlit:

<http://cours-str-abklb.streamlit.app/>

Shiny: As a Small-Project